

Student: Andrius Klimas

D number: D00246310

Lecturer: Andrew Shaw

Subject: Deployment

Date Due: 10/05/2026

Course: System Operations and Development

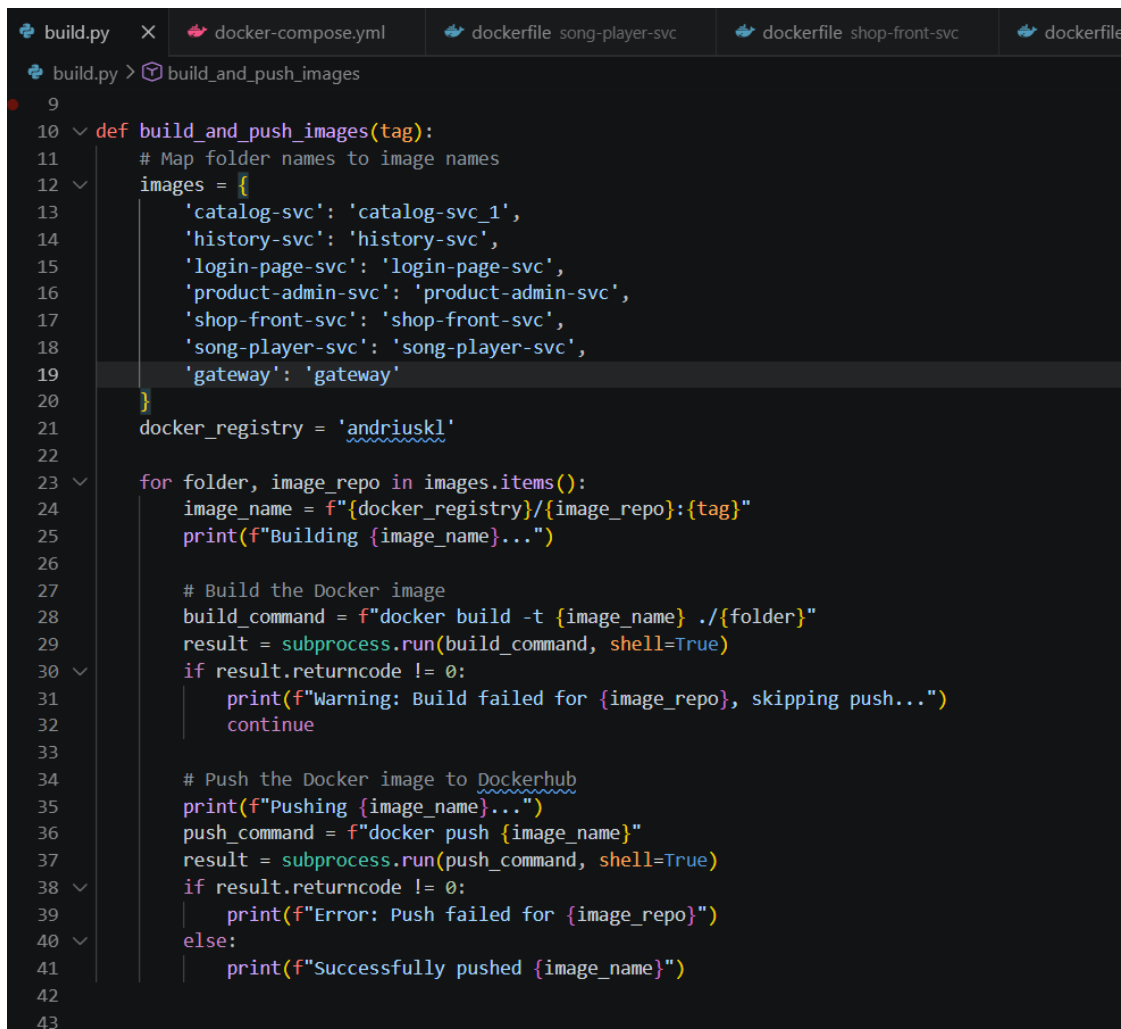
Introduction

In this document I will be going over how to upload a project to Kubernetes and azure. I will be also going over the changed needed to make this happen as well as any automation that was done to get the project fast tracked and to reduce repetitive inputs.

Automation

Pipeline

To do this I used 2 different files, they are called .gitlab-ci.yml which is created when setting up a pipeline on gitlab, and I will be using build.py.



```
build.py  X  docker-compose.yml  dockerfile song-player-svc  dockerfile shop-front-svc  dockerfile
build.py > build_and_push_images
9
10 def build_and_push_images(tag):
11     # Map folder names to image names
12     images = {
13         'catalog-svc': 'catalog-svc_1',
14         'history-svc': 'history-svc',
15         'login-page-svc': 'login-page-svc',
16         'product-admin-svc': 'product-admin-svc',
17         'shop-front-svc': 'shop-front-svc',
18         'song-player-svc': 'song-player-svc',
19         'gateway': 'gateway'
20     }
21     docker_registry = 'andriuskl'
22
23     for folder, image_repo in images.items():
24         image_name = f"{docker_registry}/{image_repo}:{tag}"
25         print(f"Building {image_name}...")
26
27         # Build the Docker image
28         build_command = f"docker build -t {image_name} ./{folder}"
29         result = subprocess.run(build_command, shell=True)
30         if result.returncode != 0:
31             print(f"Warning: Build failed for {image_repo}, skipping push...")
32             continue
33
34         # Push the Docker image to Dockerhub
35         print(f"Pushing {image_name}...")
36         push_command = f"docker push {image_name}"
37         result = subprocess.run(push_command, shell=True)
38         if result.returncode != 0:
39             print(f"Error: Push failed for {image_repo}")
40         else:
41             print(f"Successfully pushed {image_name}")
42
43
```

```

44
45
46 ∨ if __name__ == "__main__":
47 ∨     if len(sys.argv) != 2:
48         print("Usage: python build.py <tag>")
49         sys.exit(1)
50
51     tag = sys.argv[1]
52
53
54     build_and_push_images(tag,)
55
56

```

```

gitlab-ci.yml > {} build-job > {} rules > {} 0
gitlab-ci - GitLab CI Configuration file (ajson)
1 ∨ stages:
2   - build
3
4 ∨ build-job:
5   stage: build
6   script:
7     - echo "Starting build job"
8     - echo $SHELL
9     - bash --version
10    - python3 --version
11    - echo "Environment variables set"
12    - echo $DOCKER_PASSWORD | docker login -u $DOCKER_USERNAME --password-stdin
13    - python3 build.py $CI_COMMIT_TAG
14    - docker logout
15 ∨ rules:
16   - if: $CI_COMMIT_TAG
17

```

These 2 files work together to create an image of the files specified and upload them to docker hub under the account specified.

Below you can see the runners, they are currently offline as I do not have the server I am using online.

Available Runners

Create project runner

Assigned project runners2

Other available project runners2

Group

Instance

Status	Runner configuration?	Owner?
 Stale  Idle	#689 (vW66LJzV)  Project eo 2  Last contact: 2 weeks ago  Created by D00246310 3 weeks ago	Lab3_music_player_deployment   
 Stale  Idle	#687 (4IfBzCqh)  Project eo 7  Last contact: 2 weeks ago  Created by D00246310 3 weeks ago	Lab3_music_player_deployment   

Below you can see the pipeline running, it is making a image of the specific docker containers. The tag is 2.0.5

D00246310 / Lab3_music_player_deployment / Jobs / #23792

Log timestamps in UTC. Search visible log output

520 15:07:42 Building [MASKED]/catalog-svc_1:2.0.5...

521 15:07:42 Pushing [MASKED]/catalog-svc_1:2.0.5...

522 15:07:42 Successfully pushed [MASKED]/catalog-svc_1:2.0.5

523 15:07:42 Building [MASKED]/history-svc:2.0.5...

524 15:07:42 Pushing [MASKED]/history-svc:2.0.5...

525 15:07:42 Successfully pushed [MASKED]/history-svc:2.0.5

526 15:07:42 Building [MASKED]/login-page-svc:2.0.5...

527 15:07:42 Pushing [MASKED]/login-page-svc:2.0.5...

528 15:07:42 Successfully pushed [MASKED]/login-page-svc:2.0.5

529 15:07:42 Building [MASKED]/product-admin-svc:2.0.5...

530 15:07:42 Pushing [MASKED]/product-admin-svc:2.0.5...

531 15:07:42 Successfully pushed [MASKED]/product-admin-svc:2.0.5

532 15:07:42 Building [MASKED]/shop-front-svc:2.0.5...

533 15:07:42 Pushing [MASKED]/shop-front-svc:2.0.5...

534 15:07:42 Successfully pushed [MASKED]/shop-front-svc:2.0.5

535 15:07:42 Building [MASKED]/song-player-svc:2.0.5...

536 15:07:42 Pushing [MASKED]/song-player-svc:2.0.5...

537 15:07:42 Successfully pushed [MASKED]/song-player-svc:2.0.5

538 15:07:42 Building [MASKED]/gateway:2.0.5...

539 15:07:42 Warning: Build failed for gateway, skipping push...

540 15:07:42 \$ docker logout

541 15:07:42 Removing login credentials for https://index.docker.io/v1/

542 15:07:42 Cleaning up project directory and file based variables

543 15:07:42 Job succeeded

Duration: 51 seconds

Finished: 2 weeks ago

Queued: 3 minutes

Timeout: 1h (from project)

Runner: #687 (4lfBzCqh)

Source: Push

Commit 1b3b3597

Add Kubernetes deployment and service configurations for catalog, login, history, product admin, shop front, and song player services

Pipeline #5543 Passed for 2.0.5

build

Related jobs

→ build-job

Here you can see them on docker hub



Repositories

All repositories within the andriuskl namespace.



All content



Name

andriuskl/song-player-svc

andriuskl/shop-front-svc

andriuskl/product-admin-svc

andriuskl/login-page-svc

andriuskl/history-svc

andriuskl/catalog-svc_1

andriuskl/catalog-svc

andriuskl/simple-server

[Repositories](#) / [song-player-svc](#) / [General](#)

andriuskl/song-player-svc

Last pushed 15 days ago · Repository size: 385 MB · 0 · 88

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

DOCKER SCOUT INACTIVE
[Activate](#)

This repository contains 4 tag(s).

Tag	OS	Type	Pulled	Pushed
2.0.5		Image	less than 1 day	15 days

Manifests

I have 8 pairs of manifests.

Catalog-db service and deploy

Login-db service and deploy

Catalog-svc service and deploy

Shop-front-svc service and deploy

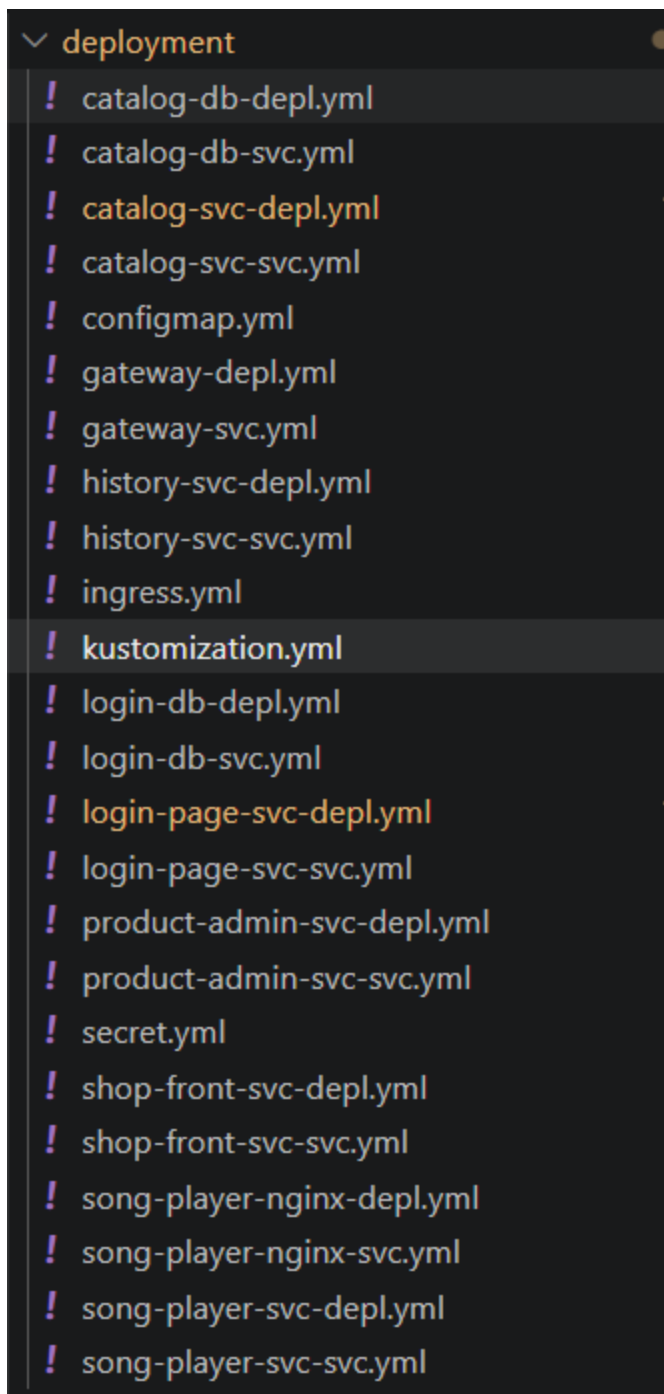
Product-admin-svc service and deploy

Song-player-svc service and deploy

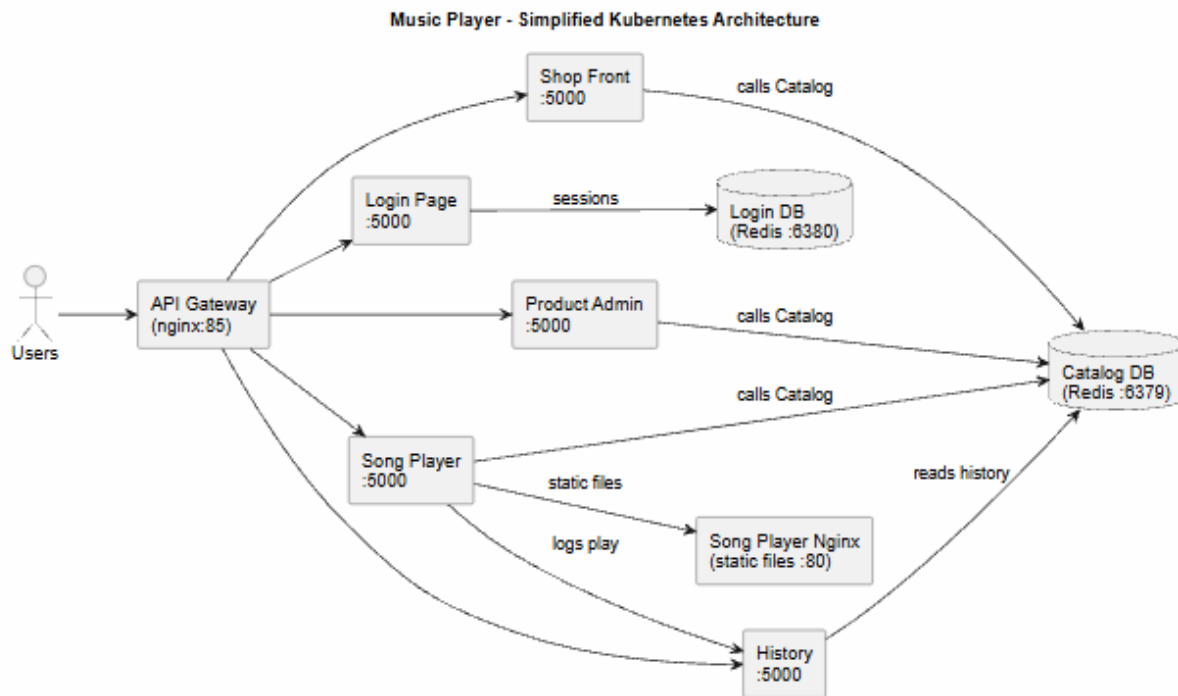
History-svc service and deploy

Login-page-svc service and deploy

Please see images below



And the image below explains how they all interact with each other



Kustomize setup

Deployment

In the kustomize file in the deployment it just has all the manifests needed to get the application to run, this includes all the 8 pairs aswell as configmap and secrets and gateway. These are used to to push these images to kubernetes


```
depl.yml    ! song-player-nginx-depl.yml    ! song-player-svc-depl.yml    ! kustomization.yml ...\deployment X
ase > deployment > ! kustomization.yml > [ ] resources > abc 17
    kustomization.yaml - Kubernetes native configuration management (kustomization.json)
1  apiVersion: kustomize.config.k8s.io/v1beta1
2  kind: Kustomization
3  namespace: music-app
4
5  resources:
6    - configmap.yml
7    - secret.yml
8    - catalog-db-depl.yml
9    - catalog-db-svc.yml
10   - login-db-depl.yml
11   - login-db-svc.yml
12   - catalog-svc-depl.yml
13   - catalog-svc-svc.yml
14   - shop-front-svc-depl.yml
15   - shop-front-svc-svc.yml
16   - product-admin-svc-depl.yml
17   - product-admin-svc-svc.yml
18   - song-player-svc-depl.yml
19   - song-player-svc-svc.yml
20   - history-svc-depl.yml
21   - history-svc-svc.yml
22   - login-page-svc-depl.yml
23   - login-page-svc-svc.yml
24   - song-player-nginx-depl.yml
25   - song-player-nginx-svc.yml
26   - gateway-depl.yml
27   - gateway-svc.yml
28   # - ingress.yml
29
```

Dev

In the dev it has the version being used in the dev environment aswell as in the top it specifies where you can locate the actual manifests in the deployment folder and specifying the ingress file to use

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

```
resources:
```

- ../../deployment
- ./ingress-gateway.yml

```
images:
```

- name: andriuskl/shop-front-svc
newName: andriuskl/shop-front-svc
newTag: 2.0.5
- name: andriuskl/product-admin-svc
newName: andriuskl/product-admin-svc
newTag: 2.0.5
- name: andriuskl/catalog-svc
newName: andriuskl/catalog-svc
newTag: 2.0.5
- # - name: andriuskl/gateway
newName: andriuskl/gateway
newTag: 2.0.5
- name: andriuskl/history-svc
newName: andriuskl/history-svc
newTag: 2.0.5
- name: andriuskl/login-page-svc
newName: andriuskl/login-page-svc
newTag: 2.0.5
- name: andriuskl/song-player-svc
newName: andriuskl/song-player-svc
newTag: 2.0.5

Prod

This is the same for the kustomization file in the prod folder

```
se / prod > kustomization.yaml > resources > ...  
kustomization.yaml - Kubernetes native configuration management (kustomization.json)  
1  apiVersion: kustomize.config.k8s.io/v1beta1  
2  kind: Kustomization  
3  
4  resources:  
5    - ../../deployment  
6    - ./azure-ingress-gateway.yml  
7  
8  
9  
10 images:  
11 - name: andriuskl/shop-front-svc  
12   newName: andriuskl/shop-front-svc  
13   newTag: 2.0.5  
14 - name: andriuskl/product-admin-svc  
15   newName: andriuskl/product-admin-svc  
16   newTag: 2.0.5  
17 - name: andriuskl/catalog-svc  
18   newName: andriuskl/catalog-svc_1  
19   newTag: 2.0.5  
20 # - name: andriuskl/gateway  
21 #   newName: andriuskl/gateway  
22 #   newTag: 2.0.5  
23 - name: andriuskl/history-svc  
24   newName: andriuskl/history-svc  
25   newTag: 2.0.5  
26 - name: andriuskl/login-page-svc  
27   newName: andriuskl/login-page-svc  
28   newTag: 2.0.5  
29 - name: andriuskl/song-player-svc  
30   newName: andriuskl/song-player-svc  
31   newTag: 2.0.5
```

Screenshots of full deployment in Minkube Dashboard.

Below you can see all the services and deployments running correctly and online

```

🚢 Pulling base image v0.0.50 ...
🔧 Restarting existing docker container for "minikube" ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔧 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
💡 After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
  ▪ Using image registry.k8s.io/ingress-nginx/controller:v1.14.3
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
  ▪ Using image docker.io/kubernetes/metrics-scraper:v1.0.8
  ▪ Using image docker.io/kubernetes/dashboard:v2.7.0
🔧 Verifying ingress addon...
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

🔧 Enabled addons: storage-provisioner, dashboard, ingress, default-storageclass

❗ C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.32.2, which may have incompatibilities with Kubernetes 1.35.1.
  ▪ Want kubectl v1.35.1? Try 'minikube kubectl -- get pods -A'
🔧 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Program Files\Kubernetes\Minikube>

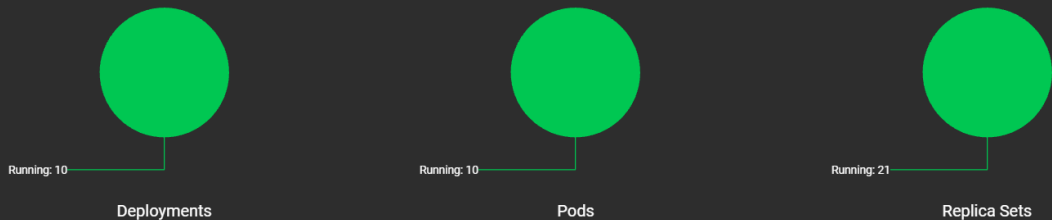
```

```

PS C:\Users\andri\OneDrive\Desktop\YEAR2SEM2\Deployment\lab-3-music-player-deployment> minikube dashboard
🔧 Verifying dashboard health ...
🔧 Launching proxy ...
🔧 Verifying proxy health ...
🔧 Opening http://127.0.0.1:54496/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...

```

Workload Status



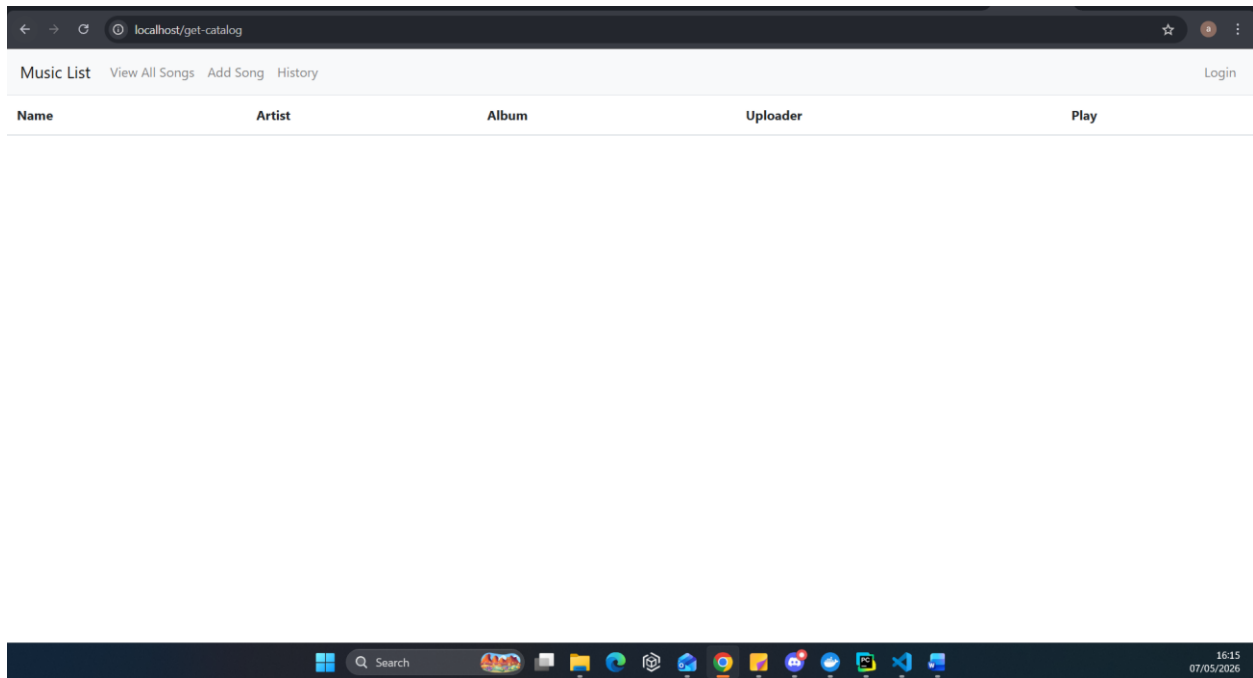
Name	Images	Labels	Pods	Created ↑
login-page-svc-depl	andriuskl/login-page-svc:2.0.5	-	1 / 1	14 days ago
song-player-nginx-depl	nginx:latest	-	1 / 1	14 days ago
song-player-svc-depl	andriuskl/song-player-svc:2.0.5	-	1 / 1	14 days ago
product-admin-svc-depl	andriuskl/product-admin-svc:2.0.5	-	1 / 1	14 days ago
shop-front-svc-depl	andriuskl/shop-front-svc:2.0.5	-	1 / 1	14 days ago
history-svc-depl	andriuskl/history-svc:2.0.5	-	1 / 1	14 days ago
catalog-svc-depl	andriuskl/catalog-svc_1:2.0.5	-	1 / 1	15 days ago
login-db-depl	redis:8.6	-	1 / 1	15 days ago
catalog-db-depl	redis:8.6	-	1 / 1	15 days ago
gateway-depl	nginx:latest	-	1 / 1	15 days ago

catalog-db	-	ClusterIP	10.109.8.63	catalog-db.music-app:6379 TCP catalog-db.music-app:0 TCP	-	15 days ago	⋮
catalog-svc	-	ClusterIP	10.109.171.3	catalog-svc.music-app:5000 TCP catalog-svc.music-app:0 TCP	-	15 days ago	⋮
gateway	-	ClusterIP	10.102.228.245	gateway.music-app:85 TCP gateway.music-app:0 TCP	-	15 days ago	⋮
history-svc	-	ClusterIP	10.105.17.3	history-svc.music-app:5000 TCP history-svc.music-app:0 TCP	-	15 days ago	⋮
login-db	-	ClusterIP	10.110.16.182	login-db.music-app:6379 TCP login-db.music-app:0 TCP	-	15 days ago	⋮
login-page-svc	-	ClusterIP	10.107.251.86	login-page-svc.music-app:5000 TCP login-page-svc.music-app:0 TCP	-	15 days ago	⋮
product-admin-svc	-	ClusterIP	10.107.164.23	product-admin-svc.music-app:5000 TCP product-admin-svc.music-app:0 TCP	-	15 days ago	⋮
shop-front-svc	-	ClusterIP	10.105.87.71	shop-front-svc.music-app:5000 TCP shop-front-svc.music-app:0 TCP	-	15 days ago	⋮
song-player-nginx	-	ClusterIP	10.99.98.225	song-player-nginx.music-app:8081 TCP song-player-nginx.music-app:0 TCP	-	15 days ago	⋮
song-player-svc	-	ClusterIP	10.97.230.23	song-player-svc.music-app:5000 TCP song-player-svc.music-app:0 TCP	-	15 days ago	⋮

Aswell you can see ingress running.

Name	Labels	Endpoints ↗	Hosts ↗	Created ↑
app-ingress	-	192.168.49.2	localhost	8 days ago

And below you can see I am able to hop onto the website via the minikube tunnel



Description of non functional testing

Non functional testing is testing how well the application will run and not testing if each feature works. So I would do the following tests:

Load test: to check even with an influx of users will the applications response time stay at an acceptable level

Stress test: similar to load test, I would send a lot of requests at the same time and check if the system crashes due to an overload of requests (adding songs)

Performance test: to check the response time of key actions such as logging in, adding a song etc

Scalability: to check if the pods automatically scale up or down depending on the traffic as it is running on Kubernetes/azure

Reliability: to check if my system recovers effectively after a pod crashes (forced crash the pod if needed)

Security test: To check if there are any security issues or any information that is given to the user that should not be e.g. other users login information

The 2 applications that can be used for this purpose are

Apache JMeter as it is an open source application that can be used by anyone and it has built in reports and can easily be integrated into the system.

Locust as it uses the python language and since the application is made using python then the integration should be easy and it is open source.

Description of deployment strategies

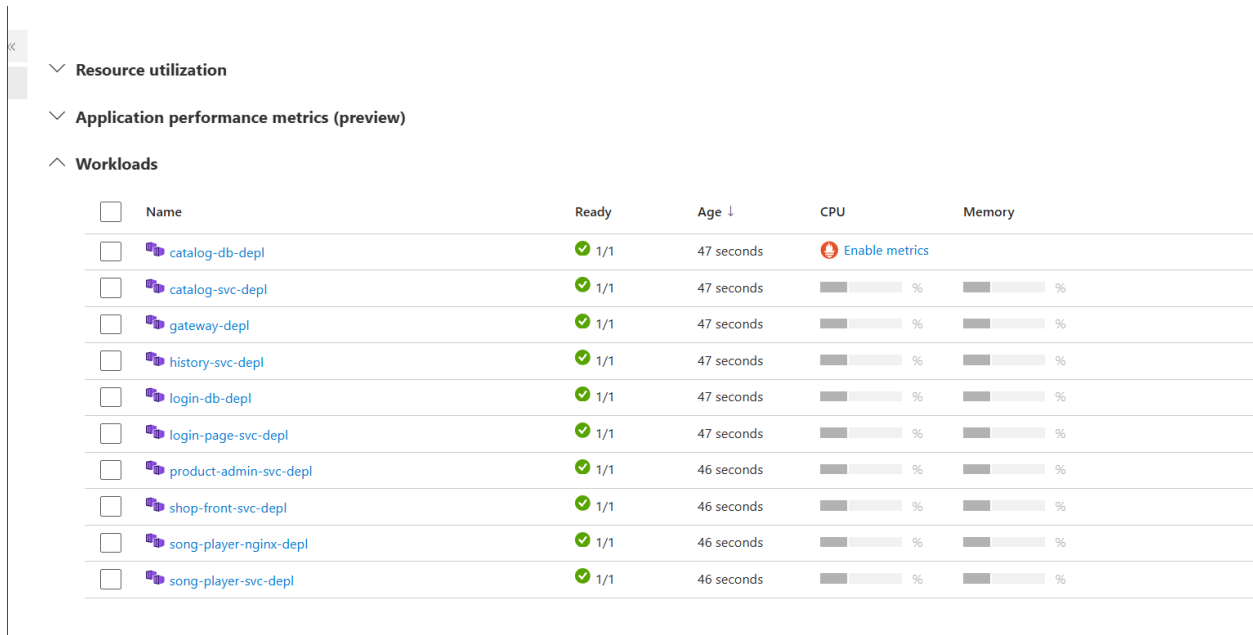
The music player uses automation in the pipeline process, where when a new version is ready to be uploaded onto docker hub, I can create a tag e.g. 2.0.6, it will create that and upload it to docker hub for me. By using Kubernetes I can control the version that is being used by changing it in the respective kustomisation file depending on the environment e.g. dev or prod. And when I push to those environments the current pods will be replaced with the new ones with almost zero down time. This will only work with individual pods, for larger scale changes we may need to implement a blue-green strategy, where we have 2 identical environments and we want to make changes on blue, we route all the traffic to green, and then start on the changes to blue, and switch over back to blue when changes are done so green can also be updated. Aswell we can test all the changes in dev before pushing them to prod so If anything happens we can just move to the newer version. And since Kubernetes is pulling from docker hub, if the newest version e.g. 2.0.6 runs into issues we can put it back to 2.0.5 and have it on a more stable version until the initial issue is located and fixed and a newer e.g. 2.0.7 version is released so it can be tested and released to prod.

Conclusion

In conclusion, this shows the automation process implemented, a brief overview of all the manifest files and how they interact with each other, the kustomization files and the different environments that there are and how they are used for each one, the project running in Kubernetes (all pods green and services running and being able to connect to it via the web), as well as an brief explanation of the non functions testing that I would do and the deployment strategies. P.S. no code was changes as we didn't add anything new into the main code.

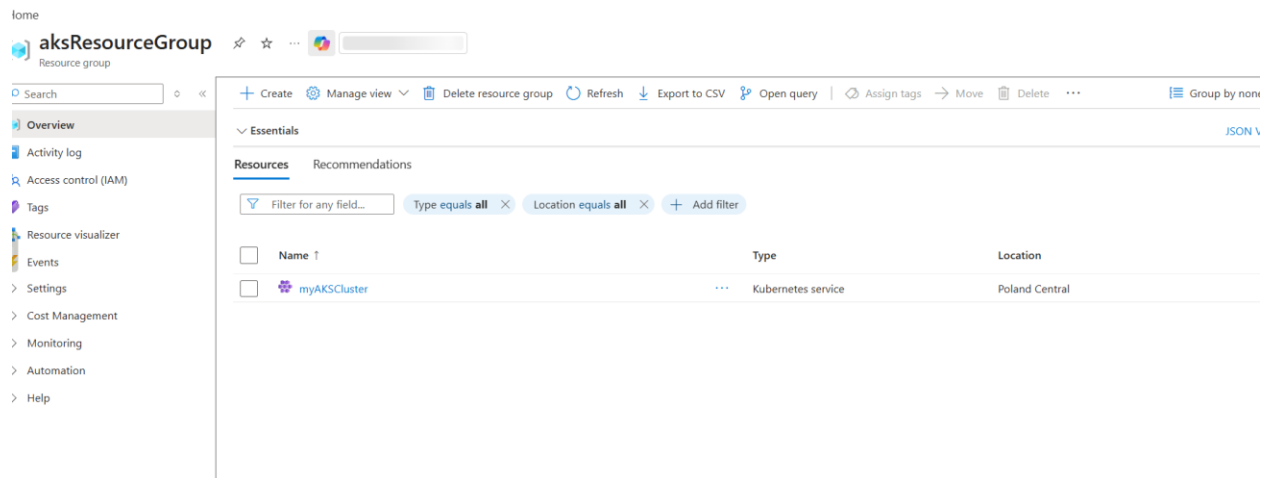
Any major issues encountered

I ran into 1 major issue when trying to get the application running onto azure, it was all added correctly, the resource group and the clusters. But when trying to connect via the ip it would give me a 503 error. I tried uploading a new ingress file but it was still the same.



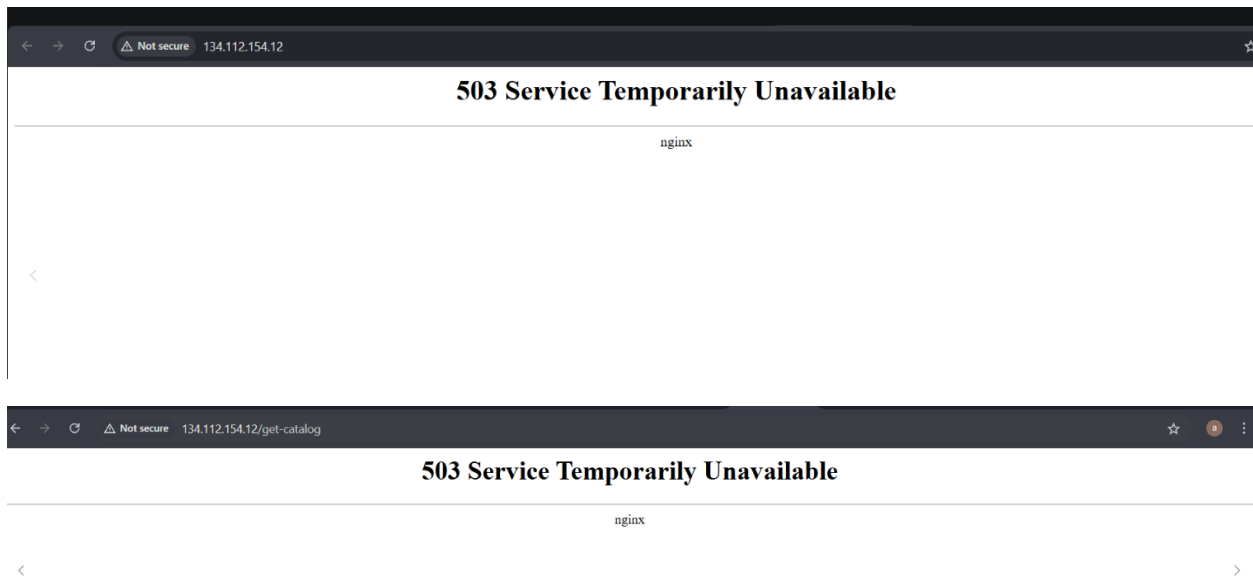
The screenshot shows the 'Workloads' section in the Azure portal. It displays a table of Kubernetes deployments within a resource group. Each row includes a checkbox, the deployment name, a 'Ready' status (green checkmark and 1/1), an 'Age' (46-47 seconds), and progress bars for 'CPU' and 'Memory' usage. A link to 'Enable metrics' is present for the first deployment.

☐	Name	Ready	Age ↓	CPU	Memory
☐	catalog-db-depl	✓ 1/1	47 seconds	Enable metrics	
☐	catalog-svc-depl	✓ 1/1	47 seconds	%	%
☐	gateway-depl	✓ 1/1	47 seconds	%	%
☐	history-svc-depl	✓ 1/1	47 seconds	%	%
☐	login-db-depl	✓ 1/1	47 seconds	%	%
☐	login-page-svc-depl	✓ 1/1	47 seconds	%	%
☐	product-admin-svc-depl	✓ 1/1	46 seconds	%	%
☐	shop-front-svc-depl	✓ 1/1	46 seconds	%	%
☐	song-player-nginx-depl	✓ 1/1	46 seconds	%	%
☐	song-player-svc-depl	✓ 1/1	46 seconds	%	%



The screenshot shows the 'Overview' page for the 'aksResourceGroup' in the Azure portal. The left sidebar contains navigation links for Overview, Activity log, Access control (IAM), Tags, Resource visualizer, Events, Settings, Cost Management, Monitoring, Automation, and Help. The main content area shows the 'Essentials' section with a 'Resources' tab. A table lists the resources in the group, including a Kubernetes service named 'myAKSCluster' located in 'Poland Central'.

☐	Name ↑	Type	Location
☐	myAKSCluster	Kubernetes service	Poland Central



Even after rerunning ingress issue persisted. I could not find the root cause to this. It is an issue with ingress but it is working correctly in Kubernetes. The only change I made was change the below

```
io.k8s.api.networking.v1.Ingress (v1@ingress.json)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "8m"
spec:
  rules:
  - host: localhost
    http:
      paths:
      - pathType: Prefix
        path: /get-catalog
      backend:
```

To

```
metadata:
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "8m"
    kubernetes.azure.com/app-routing-zone: ""
  spec:
    ingressClassName: webapprouting.kubernetes.azure.com
    rules:
    - http:
        paths:
        - pathType: Prefix
          path: "/get-catalog"
          backend:
            service:
              name: shop-front-svc
```

And it would not work, as Kubernetes is using the first one and not the second one.

URL for GitLab

<https://gitlab.comp.dkit.ie/D00246310/lab-3-music-player-deployment.git>